SIMATS ENGINEERING

ASSESSMENT TOOL 1 - Concept Mapping

COURSE NAME: Data Structures **COURSE CODE: CSA0308**

Register Number	**192572026**
Name	**Amrissh P**

COURSE OUTCOME COVERED

CO1: *Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity.* (BL2, BL3)

Assessment Tool Weightage: 35%

Questions:2 **EACH QUESTIONS CARRIES:** 10 MARKS **Total Marks:**20 **Duration:** ONE HOUR

Q1 A smart healthcare system processes large volumes of patient data such as temperature readings, medical history, and diagnostic details. Construct a concept map linking the progression from Data → Data Object → Data Structure → Abstract Data Type (ADT) → Implementation in C. Identify relevant sub-concepts such as structures, pointers, and operations, and establish meaningful relationships among them. Organize the concept map hierarchically and justify how each level contributes to efficient data organization, processing, and improved system performance.

Q2 A smart city traffic management system handles data related to vehicle movement, road networks, and signal control. Construct a concept map connecting Primitive Data Structures → Non-Primitive Data Structures → Linear → Non-Linear Structures. Include examples such as arrays, linked lists, trees, and graphs, and illustrate their applications in traffic management. Explain the relationships between these structures and justify their classification based on functionality and efficiency.

Code of Conduct:

I Amrissh P / 192572026 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Concept Mapping					
Criteria	**Weightage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

Q1. SMART HEALTHCARE SYSTEM – DATA ORGANIZATION

1. DATA

Data means the raw facts collected from patients and healthcare devices.

Examples:

- Temperature readings
- Medical history
- Diagnostic details
- Patient ID
- Blood pressure and heart rate

Relationship:

Data provides the basic information needed to create meaningful patient records.

2. DATA OBJECT

A Data Object is a meaningful representation of related data.

Examples:

- Patient
- Temperature Record
- Medical Record
- Diagnosis

For example, a patient's temperature, patient ID, date and time can be combined to form a temperature record.

Relationship:

Data Objects group related data into meaningful patient information.

3. DATA STRUCTURE

A Data Structure is a way of organizing and storing data so that it can be accessed and processed efficiently.

Array

- Stores multiple temperature readings.

- Provides quick access using an index.

Structure

- Stores different types of patient information together.
- Example: Patient ID, name, age and temperature.

Linked List

- Stores medical records dynamically.
- Pointers connect one record to another.

Tree

- Organizes diagnostic information in a hierarchical form.
- Helps in searching and classification.

Relationship:

Data Objects are organized using Data Structures for efficient storage, searching and updating.

4. ABSTRACT DATA TYPE (ADT)

An Abstract Data Type defines the operations that can be performed on data without describing the actual implementation.

Common Operations

- **Insert** – Add a new patient record.
- **Delete** – Remove an unwanted record.
- **Search** – Find a particular patient record.
- **Update** – Change existing patient information.
- **Traverse** – Visit or process stored records.

Examples

- List ADT
- Stack ADT
- Queue ADT
- Tree ADT

Relationship:

Data Structures provide the organization needed to perform ADT operations. The ADT explains **what to do**, while implementation explains **how to do it**.

5. IMPLEMENTATION IN C

The data structures and ADT operations can be implemented using the C programming language.

Structure (struct)

Used to combine different types of patient information into one record.

Pointers

Store memory addresses and are used to connect nodes in linked lists and trees.

Functions

Used to perform operations such as insertion, deletion, searching and updating.

Dynamic Memory

- malloc() – allocates memory dynamically.
- free() – releases allocated memory.

Relationship:

C functions implement ADT operations, while structures and pointers help represent and manage the data structures.

MEANINGFUL RELATIONSHIPS

Data → *forms* → **Data Objects**

Data Objects → *are organized using* → **Data Structures**

Data Structures → *support* → **ADT Operations**

ADT → *defines* → **Insert, Delete, Search, Update and Traverse**

ADT Operations → *are implemented using* → **C Functions**

Structures → *represent* → **Patient Records**

Pointers → *connect* → **Linked List and Tree Nodes**

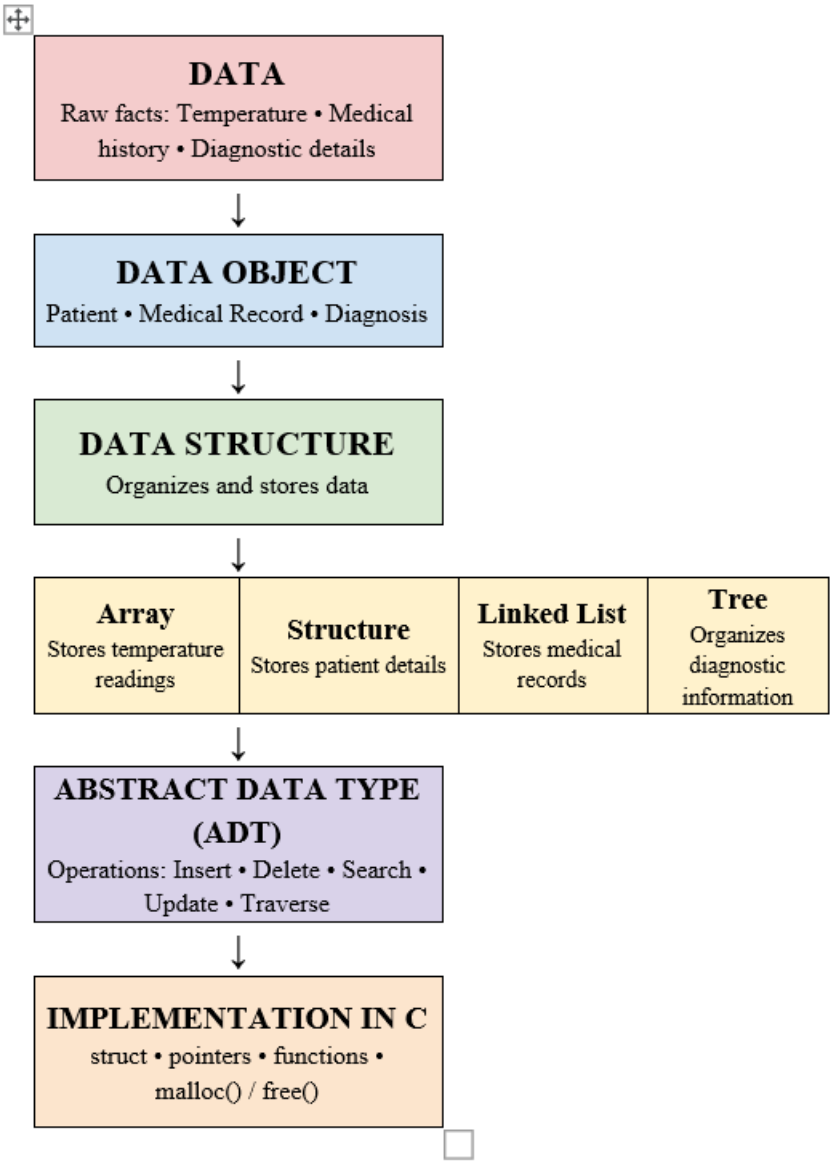
malloc() / free() → *manage* → **Dynamic Memory**

JUSTIFICATION

The progression Data → Data Object → Data Structure → ADT → Implementation in C provides a systematic way to manage healthcare information.

Data provides the raw patient information. Data Objects group related information into meaningful records. Data Structures organize these records for efficient storage and access. ADT defines the operations required to process the records. Finally, C implementation uses structures, pointers, functions and dynamic memory to perform these operations.

This approach helps the healthcare system store, search, update and process large amounts of patient data efficiently, reducing processing time and improving overall system performance.



Q2. SMART CITY TRAFFIC MANAGEMENT SYSTEM

1. PRIMITIVE DATA STRUCTURES

Primitive data structures store simple values.

Examples:

- int → vehicle count
- float → vehicle speed
- char → vehicle type
- bool → signal ON/OFF

Application: Stores basic traffic information.

Relationship: Primitive → stores → basic traffic data.

2. NON-PRIMITIVE DATA STRUCTURES

Non-primitive structures store and organize collections of data.

They are classified into:

Linear → Array, Linked List

Non-Linear → Tree, Graph

3. LINEAR DATA STRUCTURES

Array

Stores data in a sequence.

Application: Vehicle counts and signal timings.

Benefit: Fast direct access.

Linked List

Stores data using connected nodes and pointers.

Application: Vehicles waiting at traffic signals.

Benefit: Easy insertion and deletion.

4. NON-LINEAR DATA STRUCTURES

Tree

Stores data in a hierarchical form.

Application: Road hierarchy and city zones.

Benefit: Good for hierarchical searching.

Graph

Uses vertices and edges to represent connections.

Application: Roads, junctions and routes.

Benefit: Represents road connectivity and helps route planning.

5. RELATIONSHIPS

- Primitive → **stores** → Basic traffic values.
- Non-Primitive → **organizes** → Traffic data.
- Non-Primitive → **classified into** → Linear and Non-Linear.
- Linear → **includes** → Array and Linked List.
- Non-Linear → **includes** → Tree and Graph.
- Array → **stores** → Vehicle counts.
- Linked List → **manages** → Vehicle queues.
- Tree → **represents** → Road hierarchy.
- Graph → **represents** → Road networks.

6. JUSTIFICATION

Primitive structures store simple traffic values, while non-primitive structures manage large and related data. Arrays and linked lists are linear because data is arranged sequentially. Trees and graphs are non-linear because they represent hierarchical or connected relationships.

Choosing the correct structure improves storage, searching, updating and overall traffic-processing efficiency.

Smart City Traffic Management System
Vehicle movement • Road networks • Signal control

↓ uses

Data Structures
Used to store and manage traffic data efficiently

↓ classified into

Primitive Data Structures	**Non-Primitive Data Structures**
Basic data types int → vehicle count float → vehicle speed char → vehicle type bool → signal ON/OFF	Collections of related data
↓ store basic values	↓ classified into

Traffic values	**Linear Data Structures**	**Non-Linear Data Structures**
Basic values for monitoring and signal status	Sequential order	Hierarchical / network form

↓ includes

↓ includes

Array	**Linked List**	**Tree**	**Graph**
Vehicle counts Signal timings Fast direct access	Vehicle queues Changing records Easy insertion/deletion	Road hierarchy City zones Hierarchical searching	Junctions = vertices Roads = edges Route planning

↓ used for

Traffic monitoring	**Vehicle queues**	**Road hierarchy**	**Road network**
Vehicle counts and signal control	Vehicles waiting at signals	Main roads, streets and zones	Connections and route planning